

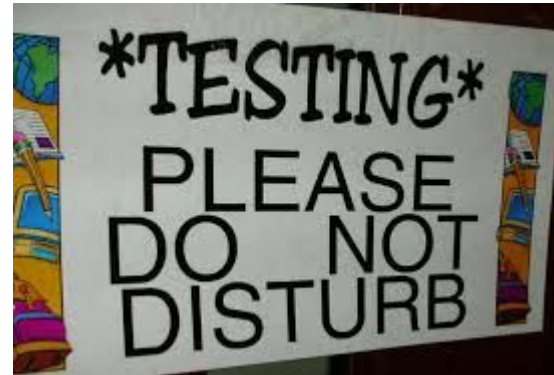
# COMS W4156 Advanced Software Engineering (ASE)

October 10, 2023



# Agenda

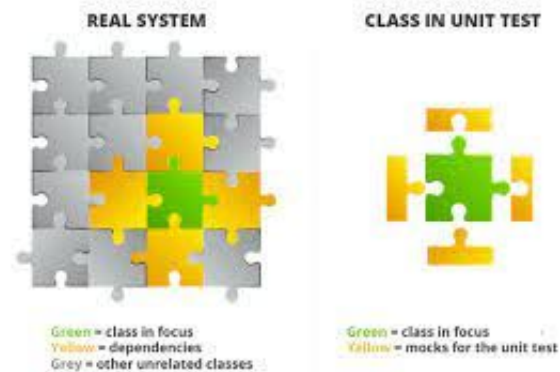
1. Class testing
2. Faking the rest of the system



# Class Testing

Class (module) testing tests that individual classes do what they are supposed to do, often conducted together with [unit testing](#) or considered part of unit testing

During unit testing of the units in a class, first **fake** all code outside each unit, then repeat unit tests using other code from the same class while still **faking** all code outside the class



# Repeating Unit Tests

```
myClass foo (myClass obj, something x) {  
    local anotherThing y;  
  
    some code;  
    y = methodInSameClass(obj,x);  
    some more code;  
    obj.field = methodOutsideClass(y);  
    code that does something;  
  
    return obj;  
}
```



During initial unit testing of foo, we should **fake** both method calls to return known values

Then during class testing, we should call the real methodInSameClass and use its real output, but **fake** the call to methodOutsideClass to return some known value(s)

Eventually during integration testing, we should also call the real methodOutsideClass

? How many times do we call the **fake** methodInSameClass and how many times do we call the **fake** methodOutsideClass ?

# Choosing Inputs for Class Testing

Equivalence partitions + boundary conditions and other techniques for choosing inputs for unit testing may, or may not, conveniently exercise all dependencies between units in the same class

Another notion of "coverage":  
exercising all direct and up to Nth degree indirect (transitive) dependencies between units in the same class



Minimize redundancy: skip any tests that would exercise the same interface (method m calling method n) with inputs equivalent to some previous passing class test (m already called n with the same or similar inputs)

# Choosing Inputs for Class Testing



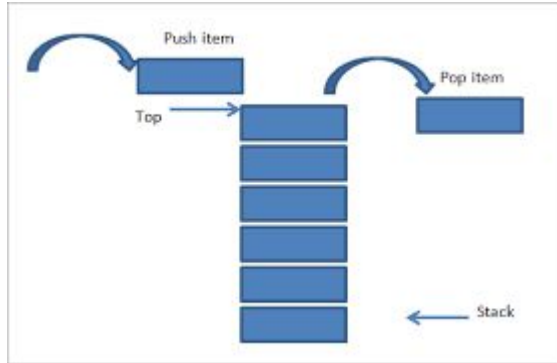
To achieve good coverage of direct dependencies, leverage conventional branch coverage in a special case of whitebox testing:

Peek inside the units to determine the execution paths whereby one unit calls another unit, and choose inputs to each calling unit that force it to call every possible callee unit

Work backwards from a call to *n* through any branches and loops to the beginning of the caller *m*, to determine the *path conditions* (constraints) on *m*'s own inputs needed to force execution to reach that call to *n*

```
myClass foo (myClass obj, something x) {  
    local anotherThing y;  
  
    some code;  
    if (something rarely true about x) {  
        y = methodInSameClass(obj,x);  
    }  
    some more code;  
    if (something often true about x) {  
        obj.field = methodOutsideClass(y);  
    }  
    code that does something;  
  
    return obj;  
}
```

# State Transition Testing



But tracing execution paths does not address cases where two or more boxes share data but neither calls the other - e.g., `push()` and `pop()` are dependent on each other because they *share* the stack data structure, but they do not normally call each other

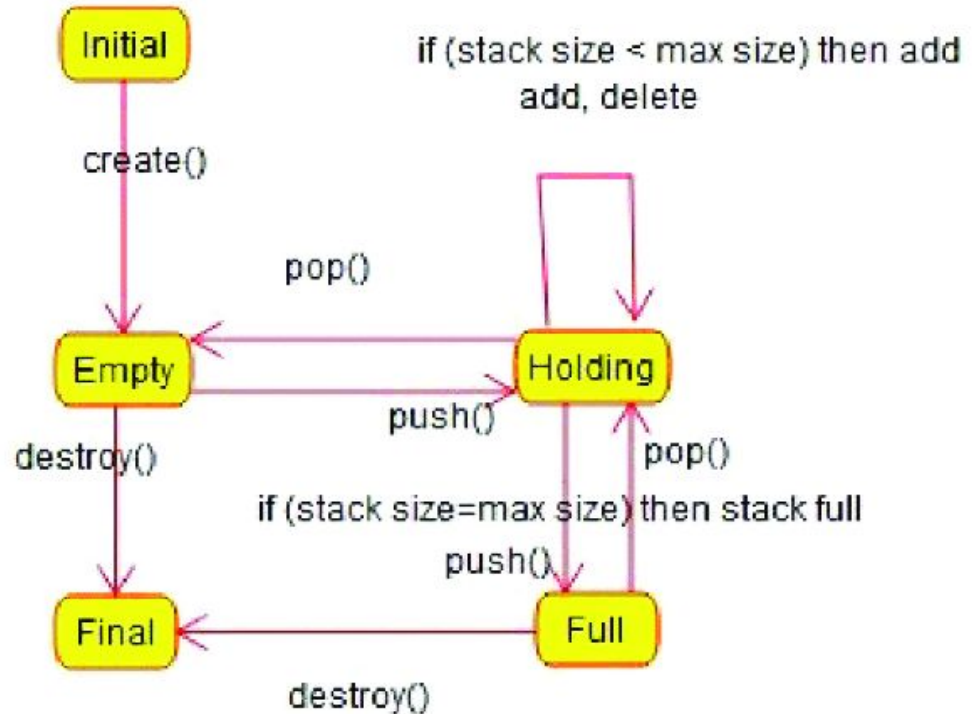
State transition testing assumes state shared among the methods of a class, so may not apply to “utility” classes that group methods that share little if any state

# Example: Testing a Stack Data Structure

Draw “[state diagram](#)” (finite state automaton) to visualize the possible states and their transitions

The idea is to choose combinations of methods and inputs to those methods that exercise all the transitions between states

There may be constraints on some transitions





# Example: Testing a Stack Data Structure

Test 1: create()

Test 2: destroy()

Test 3: create() - need new one

Test 4: push(xxx)

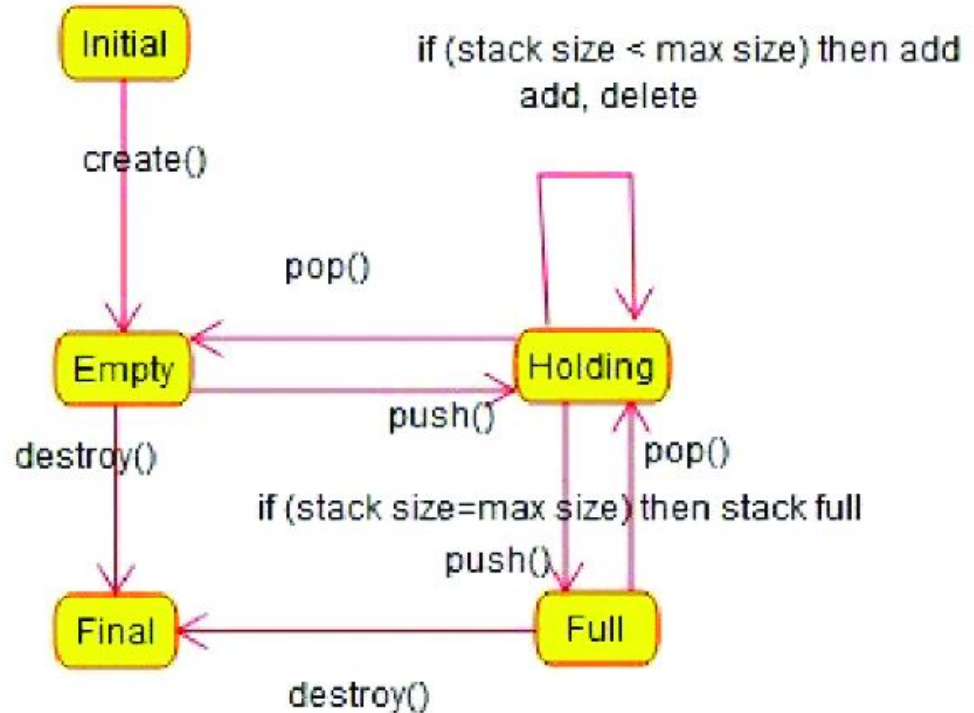
Test 5: pop() - did we get xxx?

Test 6: push(yyy)

Test 7: push(zzz)

Test 8: pop() - did we get zzz?

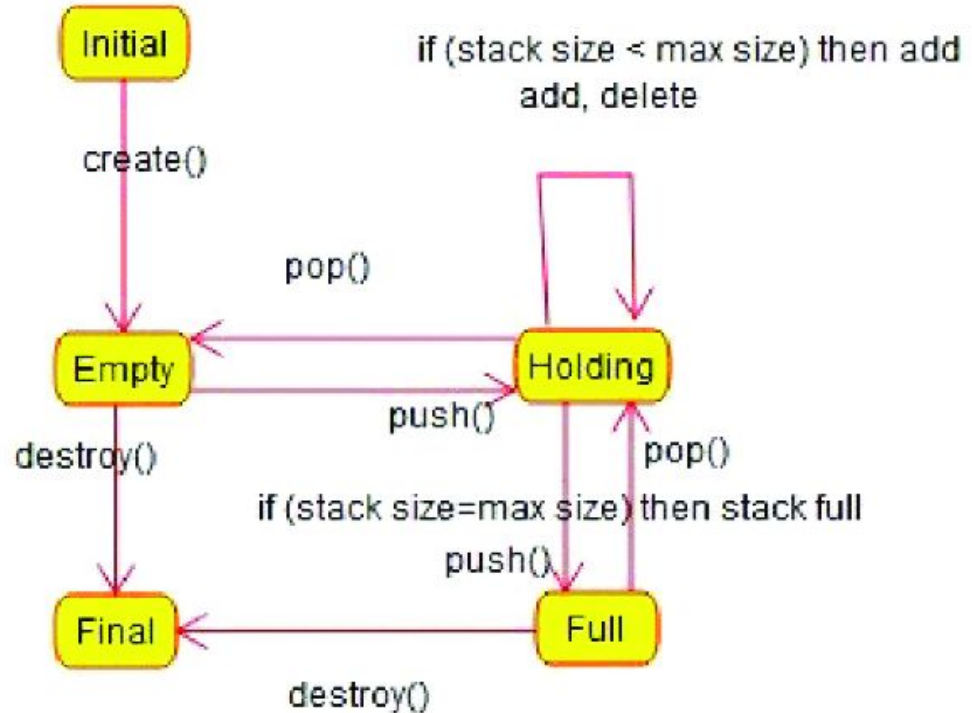
And so on...



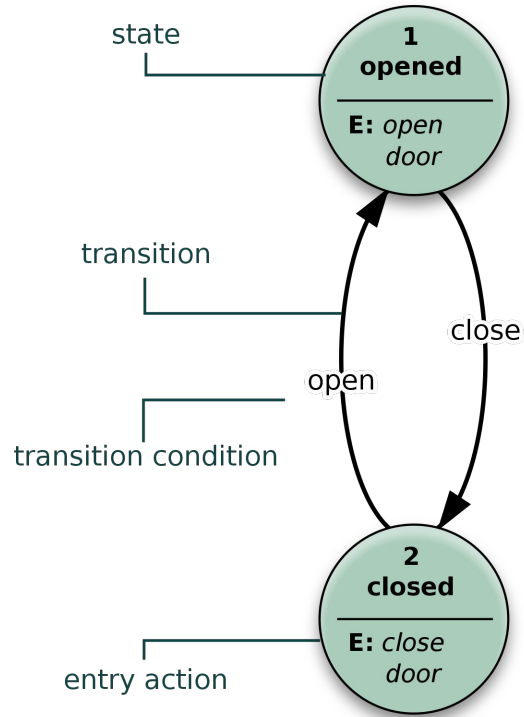
# Example: Testing a Stack Data Structure

This could go on forever, need “stopping condition”

- Follow each full path without loops
- Do each loop zero, one and two times around
- Also try invalid paths - method sequences disallowed by constraints or not included in diagram



# When to Use State Transition Testing



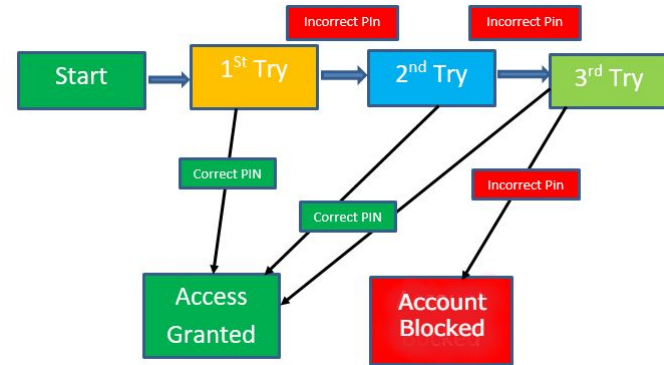
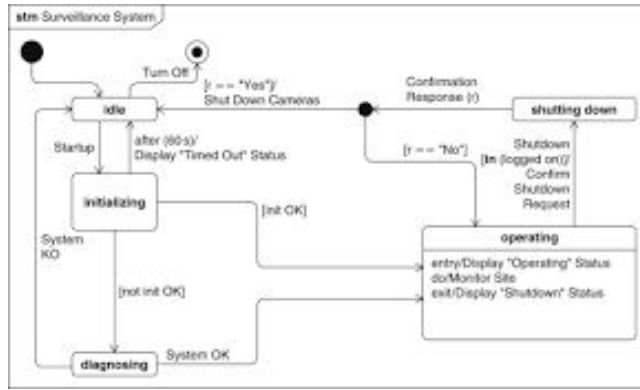
Most often applied to OO languages, where the “state” of an object is represented by its fields (and in some OO languages, its class' static fields). Thus methods that can access the same object share state

Also applicable to non-OO languages when state is scoped to modular units that can be tested in isolation from rest of program

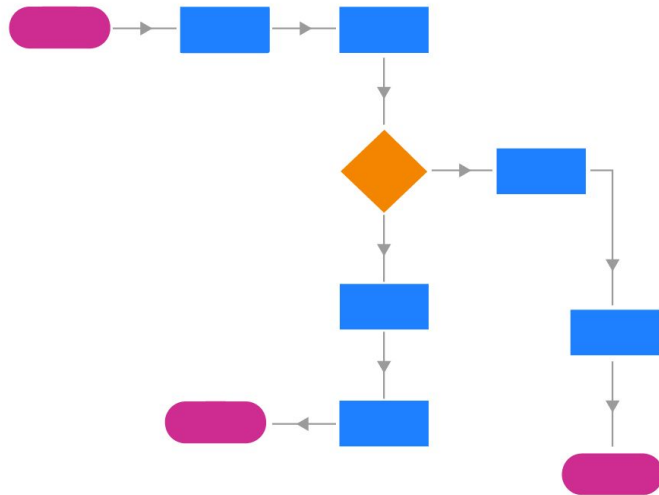
|                                                                                    |                                                                                     |                                                                                     |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| shared                                                                             | mutable                                                                             | state                                                                               |
|  |  |  |
| this value<br>can be used<br>in many parts<br>of the program                       | this value<br>can be modified<br>in place                                           | this value is<br>stored in one place<br>and can be accessed                         |

# Test All Transitions in the State Diagram

Does not scale well. ? Why ?



# State Transition Testing Also Applies to (Some) API Testing



For many (not all) APIs, the appropriate ordering among API calls follows state transitions within the API implementation

Choose combinations of entry points and inputs to those entry points that exercise all possible transitions between states

But the API state diagram may be implicit - not known to developer/tester of client code

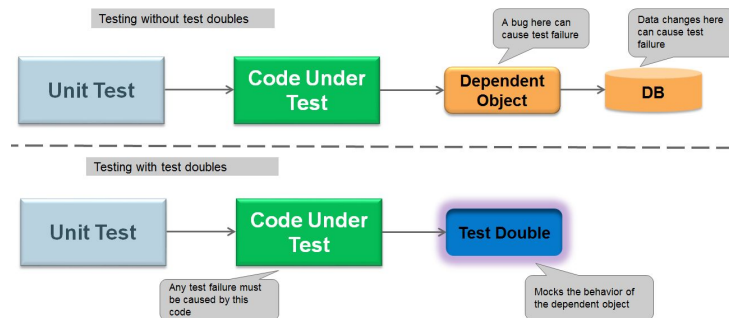
And/or the API may include unrelated utility functions that share little or no state

# Testing in Isolation

Unit testing aims to test each method and class *in isolation from the rest of the program*

The method, or a series of methods in the same class, is called from a test runner or another kind of test driver, not from the other code in the rest of the program

When the methods under test would normally call other code in the rest of the program outside the class, or otherwise interact with the environment outside the class, that other code and/or environment is **faked**



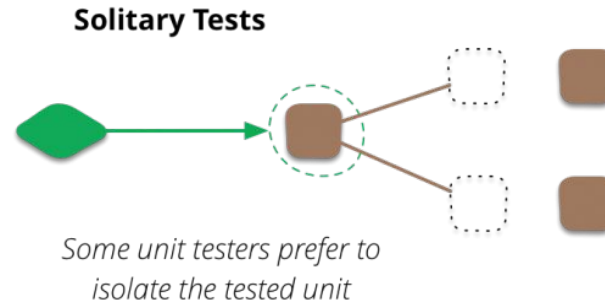
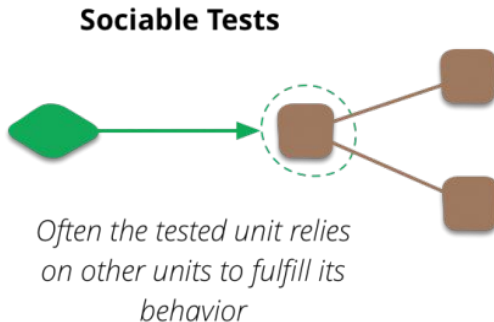
Exceptions: basic libraries like math and string processing. ? Why ?

# Sociable Tests vs. Solitary Tests

Both require a test driver, but in sociable tests the method under tests calls other code in the codebase and external resources more-or-less the same way it would during production (although usually in a separate testing environment)

Solitary tests instead call special other code (**fakes**) that stands in for that other code and external resources

Solitary tests are more expensive than sociable tests. ? Why ?



# Unit Tests with Side-Effects



Avoid when possible!

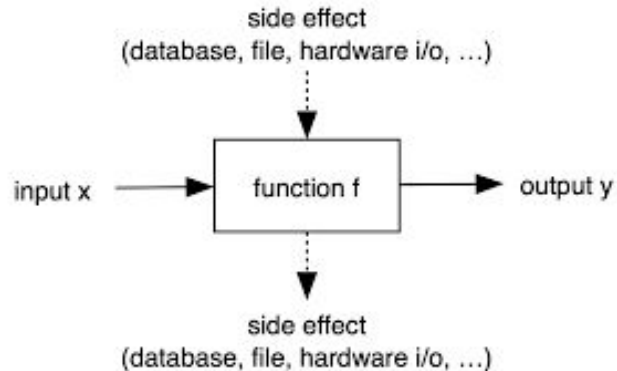
But if your units have, or might have, side-effects outside the class (i.e., beyond the object's own fields and the class' static fields), then unit testing needs to consider what to do with these outputs that change global shared state and/or produce externally visible output

Need to **fake** that global shared state and/or external visible output so it isn't really global and it isn't really externally visible

Unit tests need to be very fast, so they can run whenever the unit is changed.

Real resources often too slow

Bugs found during unit testing should be in the unit under test, not in some other code. ? Why ?





# Agenda

1. Class testing
2. Faking the rest of the system



# What Are Test Doubles?

The fakes used in testing are called “[test doubles](#)”

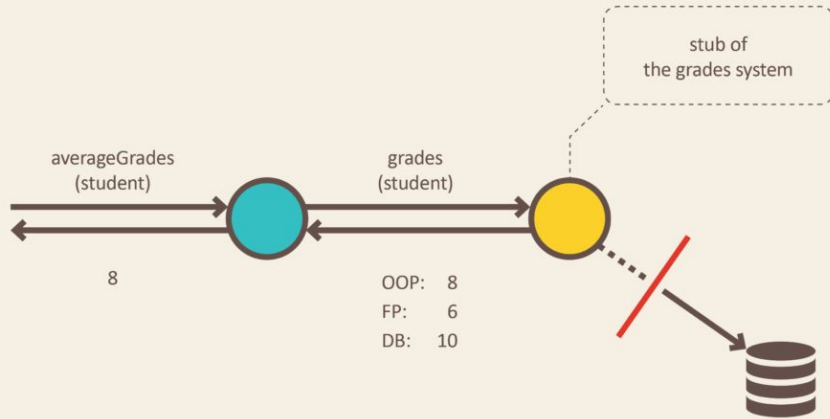
A test double is anything that replaces a production object or service for testing purposes

Analogous to stunt doubles who perform dangerous action sequences in movies



# Several Kinds of Test Doubles

## Stub



*Stubs* support *state verification*, meaning they provide canned (hardcoded) data

When called during a test, they always return the same value or always do the same thing

# Stubs

```
private class FooStub implements Foo
{
    public String bar()
    {
        return "bar";
    }
}
```

```
public class FooCollectionTest
{
    @Test
    public void it_should_return_joinedBars()
    {
        FooCollection sut = new FooCollection();
        sut.add(new FooStub);
        sut.add(new FooStub);
        assertEquals("barbar", sut.joined());
    }
}
```

A stub implements a required interface and delivers pre-defined inputs to the caller when the stub's methods are called. Stubs are programmed only for the particular test's scope and might provide inputs intended to force a specific execution path in the caller

Stubs can replace standard or third-party libraries as well as code from your codebase

Stubs can raise exceptions as well as return values. Suppose we need to simulate a hardware failure or transaction timeout, then the stub would always raise that failure or timeout exception

# Dummy Objects

A *dummy object* is a special case of a stub that does not even return a value

It's like a stunt double that does no stunts

In movies, sometimes a double doesn't perform anything; they just appear on the screen. For example, the actor's character needs to stand in a crowded place where it would be risky for the real actor to go



# Dummy Objects

A dummy object might be passed as a mandatory parameter object

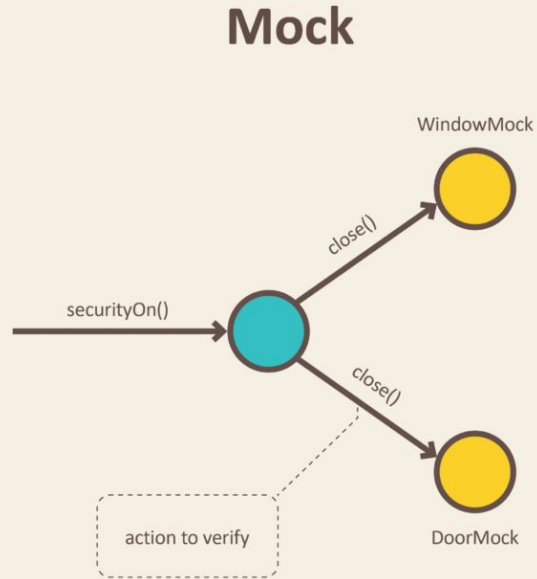
The dummy object is not directly used in the test or in the code under test, but it is required to call the code under test

Not the same as a null object - which can indeed be used, e.g., in a conditional - if this parameter is null then do x else do y

```
private class FooDummy implements Foo
{
    public String bar() {
        return null;
    }
}

public class FooCollectionTest
{
    @Test
    public void it_should_maintain_a_count()
    {
        FooCollection sut = new FooCollection();
        sut.add(new FooDummy);
        sut.add(new FooDummy);
        assertEquals(2, sut.count());
    }
}
```

# More Kinds of Test Doubles



*Mocks support **behavior verification**, meaning they actually do something*

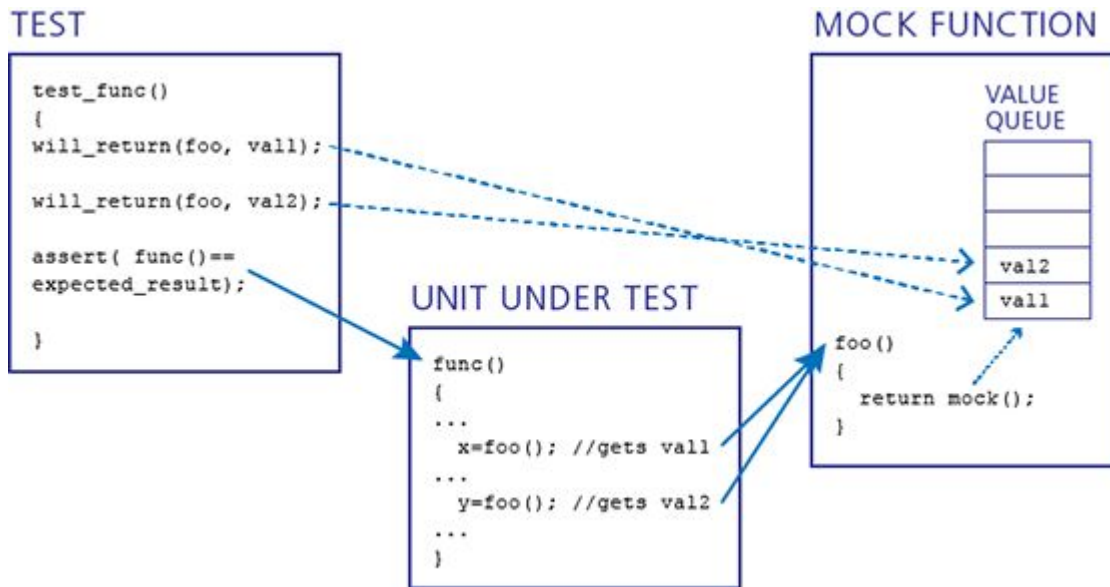
A mock generally does more than a stub

# Mocks

A mock might return hardcoded values for certain calls, but it *remembers* those calls and values

We can use mocks to verify that all expected actions are performed in the expected order with no duplicates - mocks might throw an exception when they receive an unexpected call or don't receive an expected call

*Spy objects* are a special case of mocks added to stubs just to record calls to the stub (the remembering part of mocks)





# Example

```
public class SecurityCentral {  
    private final Window window;  
    private final Door door;  
  
    public SecurityCentral(Window window, Door  
door) {  
        this.window = window;  
        this.door = door;  
    }  
  
    void securityOn() {  
        window.close();  
        door.close();  
    }  
}
```

```
public class SecurityCentralTest {  
    Window windowMock = mock(Window.class);  
    Door doorMock = mock(Door.class);  
  
    @Test  
    public void enabling_security_locks_windows_and_doors() {  
        SecurityCentral securityCentral = new  
SecurityCentral(windowMock, doorMock);  
        securityCentral.securityOn();  
        verify(doorMock).close();  
        verify(windowMock).close();  
    }  
}
```

How do we know that SecurityCentral will close the real door and the real window when it needs to? We don't - until integration testing with real resources



# Fake Objects

During integration or system testing, we need to test that SecurityCentral closes real doors and windows - without disturbing production

One way to do this is with a *fake object*

A fake object is a special case of a mock that provides a simplified and/or light-weight working implementation but is not suitable for production use, e.g., an in-memory database, a payments service without real money, a scaled-down network inaccessible by real users



# Mocks Combine Fakes with Spies on Fake Objects

```
public class FooCollectionTest
{
    @Test
    public void it_should_return_joinedBars()
    {
        // instance
        Foo fooMock = mock(Foo.class);
        // behavior
        when(fooMock.bar()).thenReturn("baz", "qux");

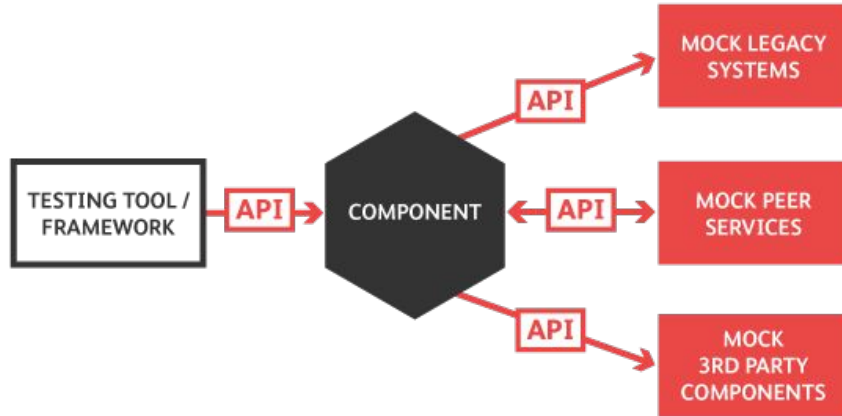
        FooCollection sut = new FooCollection();
        sut.add(fooMock);
        sut.add(fooMock);

        assertEquals("bazqux", sut.joined());
        // verify
        verify(fooMock, times(2)).bar();
    }
}
```

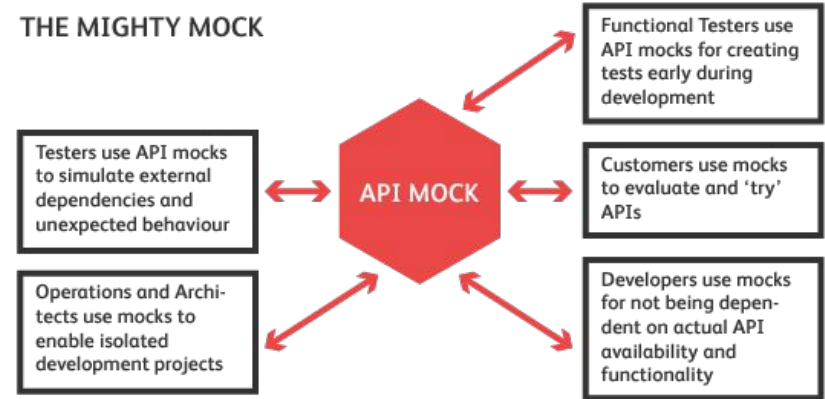
Mocking can prevent some tests from being “[flaky](#)”, i.e., passing sometimes and failing other times when nothing has changed

For example, using an in-memory database avoids timeouts and other issues that can arise with a disk-based (or over-the-network) database

# Mocks can Serve as Fake APIs



## THE MIGHTY MOCK



# Mocking Frameworks

There are mocking frameworks that work together with unit testing frameworks for most major languages

Java: [Mockito](#), [PowerMock](#), [EasyMock](#), [JMockIt](#)

C++: [gMock](#) comes with [googletest](#), [Fakelt](#), [trompeloeil](#), [mockcpp](#)

Javascript: Many [unit testing frameworks](#) have built-in mocking, e.g., JEST has built-in [Mock Functions](#) and [Manual Mocks](#)

You should use mocks for testing your team project

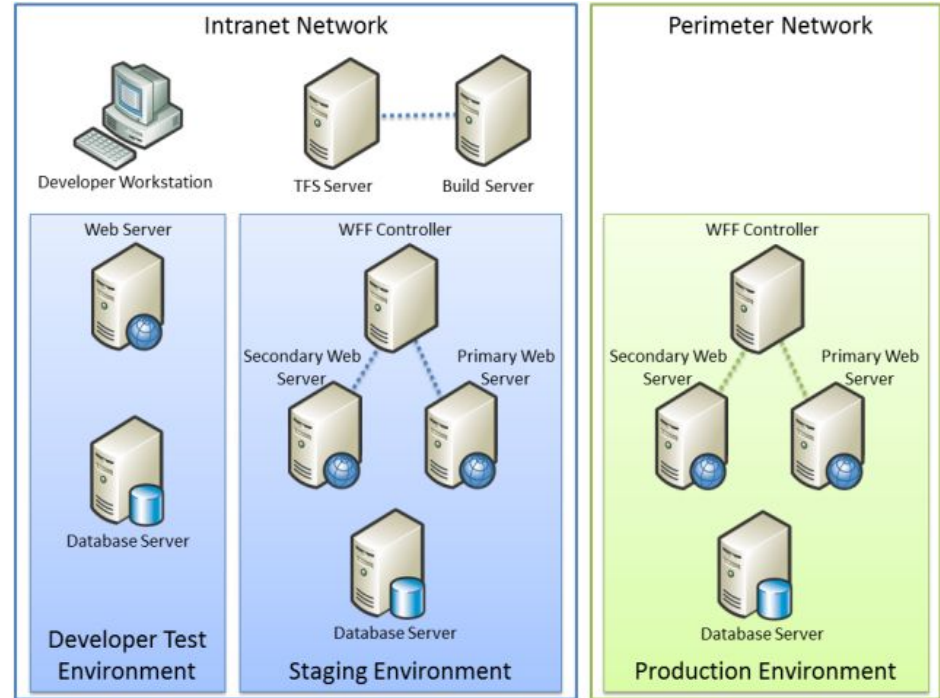


# When NOT To Use Test Doubles

End-to-end system testing should use real resources, but not usually the *production* instances of those resources

End-to-end system testing for financial services does not move real money, it does not send “final notices” to real customers, etc.

Imagine a newly hired developer using their laptop to test their modified code on the production data for amazon, netflix, tiktok, ... your account on your favorite internet service



# Upcoming Assignments

- [First Iteration](#) due Thursday, October 19, by 11:59pm
- [First Iteration Demo](#) due Thursday, October 26, by 11:59pm





# Thursday

mocking demo

integration testing



# Ask Me Anything